

CDWIA Mock Interview — 100 Questions, Beginner to Principal Engineer

How to use this

Five tiers of 20 questions, increasing difficulty. Each question has a strong reference answer and one likely follow-up. This is a study script, not a live chat — read a question, answer it out loud or in writing *before* looking at the reference answer, then compare.

Want the live version? Paste your own answer to any question back to me in chat and I'll evaluate it against the reference answer, flag weaknesses, and drill you with follow-ups the way a real interviewer would — the full evaluate → critique → stronger-answer → deeper-follow-up loop from your original brief works best interactively rather than pre-scripted for all 100.

Tiers:

- **Tier 1 (Q1-20): Foundational** — can you describe the system correctly?
 - **Tier 2 (Q21-45): Intermediate** — can you explain why each piece exists?
 - **Tier 3 (Q46-70): Advanced** — can you reason about tradeoffs and failure modes?
 - **Tier 4 (Q71-90): Expert** — can you design extensions and handle novel scenarios?
 - **Tier 5 (Q91-100): Principal** — can you defend the architecture under adversarial questioning and reason about organizational/scaling implications?
-

Tier 1 — Foundational (Q1-20)

Q1. What does CDWIA do, in one sentence? It lets business users ask natural-language questions about cloud cost data and get answers that combine SQL query results with FinOps documentation, with citations. *Follow-up: Who is the target user, and why does that matter for the design?*

Q2. What are the three possible execution paths for a query? SQL-only, document-retrieval-only, or hybrid (both, correlated together). *Follow-up: What decides which path a query takes?*

Q3. What is the control plane responsible for? Decisions: authentication, authorization, classification, and cost checks — everything that determines whether and how a request should proceed, before any expensive execution happens. *Follow-up: What's explicitly NOT the control plane's job?*

Q4. What is the data plane responsible for? Execution: running SQL against the warehouse and retrieving documents from the knowledge base, then returning raw results. *Follow-up: Why does the control plane sit before the data plane rather than beside it?*

Q5. What does the deterministic classifier do? Looks at the incoming question and decides SQL vs. document vs. hybrid, in milliseconds, without calling an LLM, for the majority of structurally obvious questions. *Follow-up: What happens when it's not confident?*

Q6. When does the LLM planner get invoked? Only as a fallback, when the deterministic classifier's confidence score falls below a threshold — not on every request. *Follow-up: Why not just use the LLM for every routing decision?*

Q7. What is the AST SQL validator? A component that parses every LLM-generated SQL string into an abstract syntax tree and checks it against a five-step decision tree (parseable, SELECT-only, allowlisted tables/columns, has a LIMIT, under a cost threshold) before allowing execution. *Follow-up: Why an AST parser instead of a regex/keyword filter?*

Q8. What are the five checks in the AST validator's decision tree, in order? Did it parse; is it SELECT-only; is every table/column allowlisted; does it have a LIMIT (inject if missing); is the estimated cost under threshold (queue async if not). *Follow-up: Which of these five checks is corrective rather than purely a reject/accept gate?*

Q9. What database role does the SQL executor use, and why? A read-only role, so that even a bug in application logic or the validator can't result in a write — the database itself is structurally incapable of it. *Follow-up: If the validator already blocks non-SELECT statements, why does the read-only role matter?*

Q10. What is hybrid retrieval? Running both BM25 (keyword) search and vector (semantic) search on the same query and combining the results, rather than relying on just one method. *Follow-up: Give an example of a query where BM25 would succeed and vector search alone would fail.*

Q11. What does reranking do, and where does it happen in the pipeline? After BM25 and vector results are fused, a cross-encoder reranker re-scores the combined candidate set for relevance, keeping only the top few most relevant chunks before they reach the synthesizer. *Follow-up: Why not just rerank everything in the corpus directly, skipping the fusion step?*

Q12. What is citation enforcement? A rule that every factual claim in the final answer must be tagged with a citation to something actually retrieved or executed — any citation that doesn't resolve to real evidence causes the answer to be rejected. *Follow-up: Why isn't prompting the model to "always cite sources" sufficient on its own?*

Q13. What do guardrails check that citation enforcement doesn't? Whether the claim attached to a citation actually, accurately reflects what the cited source says — not just whether the citation ID is real — plus a separate scan for PII in the outbound response. *Follow-up: Give an example of a citation that would pass citation enforcement but fail the guardrail check.*

Q14. What is the result cache keyed on, and why does that matter? The normalized query text combined with the requester's data scope (tenant/business unit) — not query text

alone — so a cached answer is never served across a scope boundary it shouldn't cross.

Follow-up: What would go wrong if scope were left out of the cache key?

Q15. What does the audit log record, and why must it be immutable? What was asked, what SQL ran, what was cited, model versions used, and guardrail pass/fail — needed to answer "what did the system know when it gave this answer" for compliance. It's immutable so a bug or compromised credential can't rewrite history. *Follow-up: What specifically enforces that immutability?*

Q16. Why is ingestion a separate, asynchronous pipeline? Because processing new data/documents and serving live queries have very different latency tolerances — ingestion can wait, live queries can't. Coupling them would let new-document bursts slow down user-facing queries. *Follow-up: What gets tagged on every ingested chunk, and why does it matter later?*

Q17. What happens to a hybrid query if the SQL path fails validation? The orchestrator doesn't fail the whole request — it proceeds with the document retrieval results alone, with a warning attached, so the user still gets a partial, honest answer. *Follow-up: Why is that better than failing the whole request?*

Q18. What's the difference between authentication and authorization in this system, and where does each happen? Authentication ("who is this") happens at the gateway, resolved from a token. Authorization ("can they do this") happens in the control plane's policy engine, based on role, resource, and business context. *Follow-up: Why are they kept in separate components instead of both at the gateway?*

Q19. What's the fail-closed behavior of the policy engine, and why was it chosen? If the policy engine is unreachable, every request is denied by default, not allowed — because the alternative (fail open) means an outage silently becomes a total access-control bypass, which is far worse than temporarily blocking legitimate users. *Follow-up: What's the operational cost of fail-closed, and how would you detect it happening?*

Q20. Name one cost lever this system uses to control LLM/compute spend. Any of: the deterministic classifier (avoids LLM calls for most traffic), the model router (cheaper models for cheaper tasks), the result cache (avoids recomputation), or contextual compression (fewer tokens per call). *Follow-up: Which of these would you expect to have the single biggest cost impact, and why?*

Tier 2 — Intermediate (Q21-45)

Q21. Walk through the full flow for "which team spent the most on EC2 last month." Gateway authenticates; classifier scores this high-confidence SQL (no LLM needed); SQL agent generates SQL, validates via AST, executes read-only; synthesizer produces a cited answer from the SQL result; guardrails check it; result is cached and audit-logged; response returned. *Follow-up: At what point, if any, does this touch the knowledge agent?*

Q22. Why does the classifier's confidence threshold need tuning rather than being fixed at, say, 0.9? Because the threshold trades off two costs: too low, and low-confidence misroutes get accepted as "confident enough," causing wrong answers; too high, and too much traffic pays the LLM fallback's latency/cost unnecessarily. The right value is empirical, from labeled data, not arbitrary. *Follow-up: How would you detect the threshold has drifted out of calibration over time, in production, without manually reviewing every classification?*

Q23. Why fuse BM25 and vector search results with reciprocal rank fusion instead of averaging their scores? The two systems' scores are on incomparable scales (a BM25 score and a cosine similarity mean nothing relative to each other) — RRF fuses based on rank position within each list instead, which is scale-independent and works regardless of how either system scores internally. *Follow-up: What's a scenario where RRF itself would still produce a poor result?*

Q24. What's the actual authorization boundary for SQL scope — the allowlist, or row-level security? The AST validator's allowlist is the primary boundary — it decides what's allowed to even be attempted. RLS is the critical backstop underneath it, not a replacement — a query technically in-scope by RLS could still be catastrophically expensive or violate an intent-level rule the DB role alone doesn't encode. *Follow-up: Give an example of a query that RLS would allow but the allowlist would reject.*

Q25. Why inject a default LIMIT instead of rejecting a query that's missing one? A missing LIMIT is usually an innocent generation oversight, not malicious — rejecting it hurts UX for a harmless mistake, while injecting a sane default preserves usefulness and safety simultaneously. *Follow-up: What's the risk of the injected default being set too high, and how would you know it needs lowering?*

Q26. Why queue an expensive query asynchronously instead of just rejecting it? A legitimate, correctly-scoped question can still be too expensive to run inline (e.g. a full-year aggregation) — rejecting means the valid question goes unanswered; queuing means the user still gets a real answer, just not instantly, while protecting the warehouse. *Follow-up: How would you communicate to the user that their answer is queued rather than instant, without it feeling like a broken experience?*

Q27. What's the risk of caching hybrid answers, and how is the reference implementation's approach to it a compromise? The risk is serving stale answers after underlying data changes. A flat TTL (e.g. 15 minutes) is a compromise — simple to implement, but not precise; it accepts some staleness risk in exchange for not needing to track exact data versions per cache entry. *Follow-up: Describe the more precise alternative, and what it costs to implement.*

Q28. Why does the model router use an ordered fallback chain per task instead of one fixed model for everything? Different tasks have different cost/quality tradeoffs — classification fallback can tolerate a cheap fast model, user-facing synthesis benefits from the best available model — and a fallback chain means a single provider's outage degrades

quality one tier rather than taking the system down entirely. *Follow-up: What's the risk of a fallback chain that nobody's monitoring?*

Q29. Why are merge, synthesize, and guardrail-check modeled as three distinct steps instead of one "LLM synthesizes" step? Each can fail independently and needs its own error handling and its own audit trail — collapsing them into one opaque step means one failure mode (say, a guardrail catch) is indistinguishable from another (a merge bug), which makes debugging and monitoring much harder. *Follow-up: If you had to pick just one of these three steps to add detailed tracing to first, which would you choose and why?*

Q30. Why does the SQL path run concurrently with the retrieval path in a hybrid query, rather than sequentially? There's no data dependency between them at that stage — the SQL query doesn't need retrieved documents to be generated, and vice versa — so running them concurrently is a pure latency win with no correctness cost. *Follow-up: Under what load condition could this concurrency actually hurt more than it helps?*

Q31. What's the difference between what citation enforcement checks and what the guardrail hallucination check catches? Citation enforcement verifies a cited ID is real (exists among retrieved/executed evidence). The guardrail check goes further — it verifies the claim itself accurately follows from that source's actual content, catching a real citation attached to an inaccurate paraphrase. *Follow-up: Could a system pass citation enforcement but still produce a materially misleading answer? How?*

Q32. Why redact PII rather than reject the entire answer when it's detected in an outbound response? Rejecting discards otherwise-useful information over one sensitive field; redaction preserves the value of the rest of the answer while still protecting the sensitive data — a better tradeoff for an assistant than failing closed on any imperfection. *Follow-up: Is there a scenario where rejecting the whole answer would actually be the right call instead?*

Q33. Why does the ingestion pipeline write raw content to object storage before attempting to chunk and embed it? Because chunking/embedding is a step that can fail — if raw content weren't durably stored first, a failure there would mean the source data is simply lost, not just delayed. *Follow-up: What would you do differently if object storage writes themselves were unreliable?*

Q34. How does schema/document version tagging at ingestion connect to the audit log's value? It's what makes "what did the system know when it gave this answer" answerable — the audit log references which schema/document version was active when specific chunks were retrieved and cited, tying an answer back to a specific point-in-time state of the knowledge base. *Follow-up: What would break in the audit log's usefulness if version tagging were removed?*

Q35. Why is "hybrid" the classifier's default for low-confidence, rather than "SQL" or "document"? Hybrid tends to give supplementary coverage — worst case it runs an extra path that returns nothing useful, which is a safer failure mode than confidently committing to a single path and missing half the answer for a genuinely ambiguous question. *Follow-*

up: Is there a query pattern where defaulting to hybrid would actually be worse than defaulting to one specific path?

Q36. What's the tradeoff of contextual compression before synthesis? Lower token cost and less irrelevant text for the model to potentially misuse into a hallucinated claim, at the risk of over-compressing and removing context the synthesizer actually needed for accuracy or a valid citation. *Follow-up: How would you detect that compression has gone too far, in production?*

Q37. Why does the design explicitly separate the read-only DB role from the AST validator instead of relying on the validator alone? Defense in depth — the validator is application code and can have bugs; the read-only role is enforced by the database itself, a backstop that doesn't depend on the validator (or any application logic upstream) having been correct. *Follow-up: Name one class of bug in the validator that the read-only role would NOT protect against.*

Q38. What's the risk of over-normalizing query text before hashing it for the cache key? False cache hits — two questions that are lexically similar after aggressive normalization (e.g. stemming) but semantically different get treated as the same cached answer, trading correctness for a higher hit rate. *Follow-up: What normalization is actually used in the reference design, and why is it kept deliberately light?*

Q39. Why does OPA fail closed rather than fail open on an outage — walk through the actual cost/benefit. Fail open means an outage silently authorizes everything — a total access-control bypass exactly when there's the least visibility into what's happening. Fail closed means legitimate users are blocked, which is a real but bounded, detectable cost (a spike in denials), versus an undetectable security incident. *Follow-up: How would you specifically design monitoring to make a fail-closed outage immediately visible to on-call?*

Q40. Why is the SQL cost estimator based on EXPLAIN output rather than a row-count check? Row count doesn't capture actual execution cost — a query returning few rows can still be catastrophic if it involves a full scan or an unindexed join. EXPLAIN reflects the query's real planned execution cost. *Follow-up: What would you do if the warehouse's EXPLAIN estimates were themselves unreliable (e.g. stale statistics)?*

Q41. Describe a scenario where the deterministic classifier and the LLM planner would disagree, and explain what happens. E.g., "our storage cost increased by 45%, how do we optimize it" scores ambiguous (0.62 hybrid confidence) — low enough to trigger the LLM fallback, which then confirms hybrid based on deeper language understanding of the "what happened + what to do" pattern. The LLM's decision wins, but only because it was invoked in the first place due to low classifier confidence. *Follow-up: What if the LLM planner had said "SQL" instead of "hybrid" — would that be a bug or expected behavior?*

Q42. What's the actual difference between the gateway's WAF-style filter and the AST validator, and why does that distinction matter? The gateway filter is a coarse, surface-level backstop against obviously malicious payloads before they reach anything expensive. The AST validator is the real security boundary for SQL specifically, operating on parsed structure rather than surface text — the gateway filter being bypassed shouldn't matter

because the validator doesn't depend on it having worked. *Follow-up: Should the WAF filter and the AST validator ever share logic or a rule set? Why or why not?*

Q43. Why does rate limiting happen per-user rather than globally? A global limit lets one noisy user degrade the experience for everyone else; per-user limiting isolates blast radius — the cost is needing to track state per identity, centralized (e.g. Redis) once there's more than one gateway replica. *Follow-up: How would you rate-limit fairly across users while still protecting the system from an aggregate traffic spike across many well-behaved users?*

Q44. What's the relationship between the semantic layer and the SQL allowlist — are they redundant? No — the semantic layer governs what a metric *means* (which tables/columns "cost" maps to, ensuring consistency), while the allowlist governs what's *authorized* to be touched at all. One solves correctness of meaning, the other solves security of scope; you need both. *Follow-up: Could a query pass the allowlist but still be semantically wrong due to a semantic layer gap? Give an example.*

Q45. Why does the model router re-raise the last exception if every model in a fallback chain fails, rather than returning an empty/default response? Because silently returning nothing hides the failure from the caller, who is responsible for deciding how to degrade gracefully (a stripped-down response, a clear "temporarily unavailable" message) — that's a decision that belongs at the orchestration layer, not buried inside the router. *Follow-up: What should the orchestrator actually do when it receives that exception, for a hybrid query specifically?*

Tier 3 — Advanced (Q46-70)

Q46. Design the circuit breaker's half-open state in detail — what should it actually test? It should allow exactly one (or a small bounded number of) trial request(s) through after the cooldown — critically using a cheap, known-safe probe (e.g. `SELECT 1`) rather than an arbitrary real query, so neither a lucky fast query nor an unlucky slow one gives a false read on whether the dependency has actually recovered. *Follow-up: What should happen to the cooldown window if the half-open trial fails again?*

Q47. If you needed sub-second data freshness instead of the current async ingestion model, what breaks and how would you redesign? The core assumption "ingestion can tolerate delay" breaks. You'd likely split hot recent data (a fast, streaming-updated path) from warm/cold historical data (still on the OLAP warehouse), with the semantic layer routing by time range — adding real complexity (two systems to keep consistent) the current design avoids by accepting near-real-time freshness instead. *Follow-up: What happens to the cache and audit log design under this change?*

Q48. Design a mechanism to detect that the classifier is systematically wrong for a specific new query pattern, without waiting for manual review. Log every case where the LLM planner overrides the classifier, then periodically cluster those override cases (by embedding similarity or keyword pattern) — a recurring cluster with a consistent LLM verdict is a strong, automatable signal to retrain or extend the classifier's rules for that

pattern. *Follow-up: How would you avoid this clustering process itself becoming a source of noisy, low-value retraining signals?*

Q49. Compare "AST validation + allowlist + read-only role + RLS" against a fully sandboxed ephemeral-replica-per-query design. What does each protect against that the other doesn't? The layered design protects against malicious/buggy SQL reaching data it shouldn't, cheaply, per-query. Ephemeral replicas additionally isolate resource contention (a runaway query can't affect other tenants' queries) but at real infrastructure cost and latency, and still need the same authorization checks — an ephemeral replica with no RLS is just as leaky as a shared one. *Follow-up: Under what traffic/tenancy profile would the ephemeral-replica cost actually be justified?*

Q50. Design the golden dataset evaluation pipeline. What should it test, and how do you prevent it from going stale? At minimum: classifier routing accuracy, SQL generation correctness (against expected results, not just "did it execute"), retrieval precision@k, and end-to-end citation validity. Run it in CI on every prompt/model/schema change. Staleness risk is real — it needs ownership and an update cadence, ideally seeded from real, manually-verified production queries. *Follow-up: How would you weight regressions in this pipeline — is a classifier regression as severe as a citation-validity regression?*

Q51. A tenant has very little indexed content. What happens to their retrieval quality, and how do you address it without breaking multi-tenant isolation? Small candidate pool for fusion/reranking — precision suffers because there's genuinely less relevant content, not because the pipeline is broken. Fix without breaking isolation: a shared, tenant-agnostic corpus of universal content (e.g. AWS Well-Architected guidance) indexed separately and merged into every tenant's results, distinct from tenant-private namespaces. *Follow-up: How would you prevent the shared corpus from ever accidentally becoming a vector for cross-tenant data leakage?*

Q52. Under what load condition does running SQL generation and retrieval concurrently in the hybrid path actually hurt more than it helps? If underlying compute (connection pools, embedding API rate limits, Lambda concurrency) is shared and constrained, concurrent execution doubles simultaneous resource pressure versus sequential — under high load this could cause more queuing/throttling than the latency it saves. *Follow-up: How would you detect this specific failure mode in production metrics, distinct from general overload?*

Q53. Design the instrumentation for cost-per-query broken down by SQL/RAG/hybrid path. What's genuinely hard about getting this right? Every model call and SQL execution needs to emit cost/token data tagged with request ID and path. The hard part: hybrid queries genuinely span both SQL and RAG cost — forcing an artificial split between them is usually less useful than modeling "hybrid" as its own first-class cost category from the start. *Follow-up: How would you present this to a FinOps stakeholder who wants "the cost of the RAG system" as a single number?*

Q54. Why might RRF still produce poor results even though its math is scale-independent? RRF is scale-independent across the lists it fuses, but still sensitive to each

list's individual quality — if BM25 returns near-random results for a query with no exact-match terms, RRF still weights those noisy "top" results at their rank position, diluting genuinely relevant vector results. *Follow-up: How would you make the fusion weighting dynamic based on each list's apparent quality for a given query?*

Q55. Design a mechanism to detect embedding drift automatically, before users notice degraded relevance. Maintain a small, stable canary set of query→expected-chunk pairs, re-run on a schedule, track precision@k — a sudden drop signals either an unaccompanied embedding model change or index corruption, faster than waiting for aggregate user-facing metrics to visibly degrade. *Follow-up: How would you distinguish "embedding drift" from "the corpus content itself changed and made the canary queries stale"?*

Q56. If the SQL cost estimator itself becomes unavailable, should it fail open or closed? Justify it against the OPA precedent. Fail closed (queue everything, execute nothing inline) — same logic as OPA: the estimator's whole purpose is preventing warehouse-locking queries from running unchecked, and "assume everything is safe" during its own outage is exactly the scenario it exists to prevent. The cost of over-queuing during the outage is far smaller than one catastrophic query executing unchecked. *Follow-up: How is this different from the SQL executor's circuit breaker — shouldn't that already cover this case?*

Q57. How would you extend the AST validator to catch cross joins missing an ON clause, and what's the risk of getting the rule wrong? Walk `Join` nodes in the parsed tree, flag missing or trivially-true `on` conditions. Risk: false positives on rare-but-legitimate deliberate cross joins. Mitigation: let the cost estimator serve as a second, independent net — a deliberate small cross join is cheap and passes cost checks fine, while an accidental large one is caught by cost regardless of whether the join-condition rule flags it. *Follow-up: Would you make this new check a hard reject or a warning-plus-cost-check? Justify the choice.*

Q58. The reranker becomes a latency bottleneck under load. Rank your mitigation options by implementation cost. Cheapest: shrink candidate set size before reranking. Next: cache rerank scores for repeated (query, chunk) pairs. More involved: a faster reranker for latency-sensitive traffic, best reranker for a cached background refine pass. Most involved: batch rerank requests across concurrent queries if the service supports it. *Follow-up: Which of these would you implement first in a real incident, versus which would you plan for next quarter?*

Q59. Design a canary/shadow deployment strategy for a new model version in the router's fallback chain. Shadow-call the new model alongside production (logged, not returned to users), score it against citation validity/guardrail pass rate/sampled quality comparison over a real traffic sample and time window, then promote to a low-percentage live canary, then full production — never cut over on shadow metrics alone. *Follow-up: What's the minimum sample size/time window you'd want before trusting shadow metrics enough to promote to canary?*

Q60. What's the risk of over-compressing context before synthesis, and how would you detect you've gone too far? Removing context the synthesizer actually needed for accuracy or a valid citation — detected as a rise in citation-enforcement or hallucination-guardrail failures correlating with a compression-aggressiveness change, even as token cost drops. Treat compression tuning as a two-metric optimization (cost down, quality flat-or-up), never single-metric. *Follow-up: How would you set up an automated gate that would block a compression change from shipping if it regressed quality?*

Q61. Design a safe rollout strategy for adding a new table to the SQL allowlist. Stage in a non-production allowlist config, run the golden dataset extended with cases exercising the new table, review the diff of what SQL the generator now produces (a new table in scope can change ambiguous-question resolution, not just add capability) — only promote after review, treated like an IAM policy change with sign-off. *Follow-up: What's the specific risk of a new table changing how ambiguous questions get resolved, with a concrete example?*

Q62. Two business units need different SQL safety policies. Extend the design without duplicating validator logic. Scope the allowlist per business unit/tenant —

`SchemaAllowlist` becomes a function of the requester's `Principal` (already carrying `business_unit`), loaded per-scope. The validator's decision-tree logic itself never changes, only which allowlist instance it checks against. *Follow-up: What if the two business units also needed different LIMIT defaults or cost thresholds — does that fit the same pattern?*

Q63. Justify, to a security board, why read-only role + RLS is sufficient without per-query manual approval for cost/billing data — and where you'd concede human approval is warranted. Automated layered defense provides consistent, always-on, testable protection a human reviewer can't match at volume for low-individual-risk read-only queries. Concede human review for: any path that could plausibly touch writes (shouldn't exist), queries the cost estimator flags far beyond the async threshold (possible probing signal), and any change to the allowlist/RLS policy itself. *Follow-up: How would you quantify "far beyond the async threshold" in a way a security board would find concrete rather than hand-wavy?*

Q64. Design the state transition logic for the model router's fallback chain under a partial degradation (the top model is slow, not fully down). Latency-triggered fallback, not just error-triggered — track rolling p95 latency per model in the chain; if it crosses a threshold that would visibly degrade user experience, treat that as equivalent to a failure for routing purposes, falling to the next model, even though the top model would eventually respond successfully. *Follow-up: How do you avoid flapping between models if latency is borderline and noisy rather than clearly degraded?*

Q65. How would you extend the ingestion pipeline to support a new document type (e.g. video transcripts) without touching the retrieval path? Extend the ingestion pipeline's chunker/embedder for the new format's specifics, tag it with its own schema/document version the same way existing types are — because the retrieval path consumes chunks + embeddings through the same interface regardless of source type, it requires no changes. *Follow-up: Would video transcripts need a different chunking strategy than text documents? Why?*

Q66. What's the actual failure mode if the classifier's confidence scoring itself becomes miscalibrated (e.g. always reports 0.95 regardless of actual certainty)? The LLM fallback would almost never trigger, since confidence never drops below threshold — misroutes that should have been caught by fallback go straight through uncorrected. This would show up as a rise in downstream failures (SQL rejections, low-relevance retrieval) without a corresponding rise in fallback invocation rate — a mismatch between those two metrics is the diagnostic signal. *Follow-up: How would you build an automated check specifically for classifier confidence miscalibration, distinct from accuracy?*

Q67. Design monitoring that would catch a slow, silent degradation in SQL generation quality after a model provider's silent model update. Continuous golden-dataset evaluation, run on a schedule (not just on deploys) against the *live* production model, since provider-side model updates happen outside your own deploy cycle — a drop in golden-dataset pass rate with no corresponding change on your side is the signature of this specific failure. *Follow-up: How would you distinguish a provider-side silent update from your own prompt/schema drift causing the same symptom?*

Q68. The design uses `Protocol` interfaces throughout. What's the actual cost of this abstraction, and when would you NOT want it? Cost: indirection — every concrete implementation is one layer removed from where it's used, which can make tracing actual behavior harder for someone new to the codebase, and it's overhead not worth paying for a component you're confident will never have more than one implementation. Skip it for genuinely single-implementation, low-risk-of-change components; use it where backend swaps or multi-tenant variation are realistic, as with the retrieval backends and the allowlist. *Follow-up: Name one component in this design where you'd argue the interface abstraction ISN'T worth its cost.*

Q69. How would billion-row billing data change your SQL generation prompt design, beyond just warehouse choice? The prompt/schema documentation given to the SQL generator needs to strongly bias toward including partition-key (date) filters by default — a model with no awareness of partition pruning will generate technically correct but catastrophically expensive full-table-scanning SQL at this scale, even before the AST validator's cost check catches it. *Follow-up: Should partition-key filtering be enforced at the SQL generation prompt level, the validator level, or both? Justify.*

Q70. Design a strategy to prevent the audit log itself from becoming a bottleneck or single point of failure at high query volume. Write audit records asynchronously to a durable queue (not synchronously to the final store) before the response returns, so audit durability doesn't block response latency — but "durably queued" must happen before the response, not "written to final storage," which is a subtly different and weaker guarantee worth being precise about in an interview. *Follow-up: What's the actual difference between "durably queued" and "written to final storage," and why does that distinction matter here specifically?*

Tier 4 — Expert (Q71-90)

Q71. Design a complete incident response runbook for "cost-per-query spiked 5x overnight with no traffic increase." Order of investigation: (1) classifier fallback rate — did more traffic silently shift to the expensive LLM path; (2) cache hit rate — did a TTL/key-normalization bug tank reuse; (3) model router tier selection — is the fallback chain defaulting to a more expensive tier than expected, e.g. due to a "cheap" model's outage; (4) cost-per-query breakdown by path (SQL/RAG/hybrid) to isolate which path drove the spike. Each of these is cheap to check and rules out or confirms a specific, common cause before escalating to a deeper investigation. *Follow-up: If all four come back clean, what's your next line of investigation?*

Q72. A customer reports the system gave a confidently wrong cost figure with a valid-looking citation. Walk through your full investigation, referencing every relevant defense layer. Pull the audit log entry for that exact request: check the SQL actually executed (post-validation) against what was intended, check whether the cited source's actual text supports the claim (this is precisely what the guardrail hallucination check should have caught — did it run, did it pass, and if it passed incorrectly, why), and check the schema/document version in effect at that time in case the underlying data has since changed and the answer was correct *then*. This traces through validator → citation enforcement → guardrail → audit, in that order, because each is a distinct point where the failure could have originated. *Follow-up: Suppose the guardrail DID run and DID pass, but the claim was still wrong. What does that tell you about the guardrail's own reliability, and what would you do next?*

Q73. Design a multi-region disaster recovery strategy for this system, and state your RPO/RTO targets with justification. Cross-region backups: point-in-time recovery for the warehouse, index snapshots for the vector store, replicated audit log. Target RPO ≤15 min (bounded by how stale a warehouse replica/backup can be before a failover meaningfully loses committed billing data) and RTO ≤1 hour for the data plane (control plane, being stateless and lower-throughput, can often fail over faster). The audit log's immutability requirement adds a wrinkle — DR failover must not create a window where audit writes are lost or duplicated. *Follow-up: What's specifically hard about DR for the vector store compared to the relational warehouse?*

Q74. Design a chaos-engineering test plan specifically for this architecture's failure-handling claims (circuit breakers, fallback chains, fail-closed policy). Inject: (1) vector store latency/errors — verify hybrid degrades to SQL-only, not full failure; (2) policy engine unavailability — verify requests are denied, not allowed, and this is alerted on; (3) top-tier model outage — verify the router falls back and answer quality (not just availability) is tracked through the incident; (4) warehouse slow-query injection — verify the circuit breaker opens and subsequent requests fail fast rather than queuing behind it. Each test verifies the *specific claimed degradation behavior*, not just "did the system survive." *Follow-up: Which of these four would you run in production (not just staging), and what would make that safe to do?*

Q75. The business wants to add a fourth query path: "forecast" (predictive, not just historical). Design how this fits into the existing classifier/orchestrator architecture. The classifier needs a new class and new lexical/training signal for forecast-style questions ("what will Q3 cost be"); the orchestrator needs a new agent (a forecast agent, likely SQL-sourced historical data feeding a time-series model) behind the same `Protocol`-style interface pattern used for SQL/knowledge agents; citation enforcement needs to handle a new evidence type (model-generated projections aren't "retrieved" the same way, so the citation/confidence framework needs to represent "this is a projection, not a historical fact" distinctly). *Follow-up: How should confidence scoring differ for a forecast answer versus a historical-fact answer, and why?*

Q76. Design an A/B testing framework to evaluate whether a new SQL generation prompt actually improves accuracy in production, not just on the golden dataset. Golden dataset catches known patterns but not real-world query diversity — run the new prompt as a shadow generator alongside production on live traffic (not executed, just generated and compared), score both against a sampled human/automated review of correctness for a subset, and only then consider a live percentage rollout with the same monitoring (SQL rejection rate, citation validity) used for any other canary. *Follow-up: What's the risk of using SQL rejection rate alone as your A/B success metric?*

Q77. Justify the choice between horizontal and vertical scaling for each major component in this system (control plane, data plane, warehouse, vector store). Control plane (Lambda-style, stateless, bursty): horizontal, scales naturally with request volume, no vertical scaling story needed. Data plane execution (SQL/retrieval agents): horizontal, same reasoning. The warehouse: primarily vertical for compute-per-query within a node, horizontal via read replicas for concurrent query volume — the two scale different dimensions of the problem. Vector store: horizontal via sharding/namespacing at scale, as covered in the billion-embedding scenario. *Follow-up: Which component in this list has the least straightforward scaling story, and why?*

Q78. Design a mechanism to safely handle a schema migration on the warehouse (a column rename) with zero query-serving downtime. Dual-write or dual-read period: update the semantic layer's mapping and the SQL allowlist to accept both old and new column names simultaneously during migration, generate SQL against whichever exists, cut the SQL generator over to the new name once the migration is confirmed complete, then remove the old name from the allowlist — the allowlist change itself is the actual point of no return, not the underlying DB migration. *Follow-up: What would break first if you skipped the dual-accept period and cut over the allowlist immediately at migration time?*

Q79. How would you design cost attribution so a specific business unit can be shown "how much does the AI assistant itself cost to serve your team," as its own line item? Every model call, SQL execution, and cache miss needs to carry the requester's `business_unit` tag through to the cost-tracking layer (same instrumentation as the SQL/RAG/hybrid path breakdown, extended with an additional dimension) — the FinOps-tool-should-be-FinOps-managed principle from the design doc applied literally, at the tenant level rather than just the system level. *Follow-up: Should cached answer reuse (a*

cache hit serving a different user in the same business unit) be attributed to the original requester, the new requester, or split? Justify.

Q80. Design the interface contract for adding a completely new cloud provider (a fourth, beyond AWS/Azure/GCP) with minimal blast radius to existing code. Extend the semantic layer's metric mappings for the new provider's schema, extend the ingestion pipeline's document sources (new provider's cost-optimization docs), and add the new provider's identifier to relevant metadata filters — critically, none of this should require touching the classifier, the AST validator's decision-tree logic, or the orchestrator, since those operate on the abstraction (a query, a table reference, a chunk), not on provider-specific knowledge. *Follow-up: What's the one place where provider-specific logic genuinely can't be abstracted away, and why?*

Q81. Design a policy (Rego) test suite structure for the OPA-based authorization layer that a security team would actually trust. Table-driven input/expected-decision test cases covering: every role × every action combination expected to be explicitly allowed or denied (not just the happy path), boundary cases (a role with no explicit rule — should default-deny), and regression cases for every past incident or near-miss, run in CI on every policy change before deployment, with the test suite itself under the same review rigor as the policies. *Follow-up: How would you structure this so a non-engineer (compliance reviewer) could still meaningfully review test coverage?*

Q82. A load test reveals the control plane's classifier Lambda has a cold-start latency spike under bursty traffic. How do you address it without abandoning the serverless approach? Provisioned concurrency for the classifier Lambda specifically (a small, predictable baseline cost) rather than switching to an always-on server, since the classifier's workload profile (many small, fast invocations) is exactly what serverless is good for — the cold start is a tuning problem, not a signal the architecture choice was wrong. *Follow-up: How would you decide how much provisioned concurrency is "enough" without over-provisioning for a burst that rarely happens?*

Q83. Design an approach to periodically re-validate that the SQL allowlist doesn't contain stale or over-permissioned entries, without requiring someone to remember to audit it manually. Automated drift detection: a scheduled job diffing the allowlist config against the live warehouse schema (flagging allowlisted tables/columns that no longer exist, or exist tables/columns that were added but never explicitly excluded/included, forcing an explicit decision rather than silent inclusion or omission) — paired with the "review like an IAM policy" process for any change, this converts manual audit discipline into an enforced, automated check. *Follow-up: Should a newly-added column default to allowed or denied until explicitly reviewed? Justify against the fail-closed principle used elsewhere.*

Q84. The synthesis model provider announces a deprecation of the current model version with a 90-day migration window. Design your migration plan. Add the replacement model to the fallback chain at a lower priority first (shadow-adjacent, gets real but limited traffic only on primary-model failure), run the golden dataset against it directly, then do a percentage-based canary promotion once quality is verified — all completed well

inside the 90-day window, with the deprecated model remaining as a fallback (not primary) until full confidence, not removed abruptly at the deadline. *Follow-up: What would you do differently if the replacement model had a meaningfully different context window size?*

Q85. Design a plan for reducing the async-queue backlog if the SQL cost threshold is set too conservatively and too many legitimate queries are being queued unnecessarily.

First, quantify it — what fraction of queued queries, if run inline, would actually have been safe (retroactive analysis against actual observed execution cost vs. the pre-execution estimate that triggered queuing)? If the estimate is systematically overcautious, recalibrate the threshold against real warehouse behavior under realistic concurrency (not just single-query EXPLAIN accuracy) rather than guessing a new number. *Follow-up: What's the risk of recalibrating the threshold based on a period of unusually low concurrent load?*

Q86. How would you design the system to support a "trusted analyst" tier that can bypass certain SQL validator restrictions (e.g. a higher row limit) without weakening the security model for everyone else? Scope the LIMIT default and possibly the cost threshold per role/principal, the same pattern as the multi-business-unit allowlist scoping (Q62) — the validator's core decision-tree logic doesn't change, only which config values it checks against for a given principal. Critically, this should NOT extend to bypassing the allowlist or the SELECT-only check — those are security boundaries, not throughput tuning, and conflating "trusted" with "can run unsafe SQL" would be a real regression. *Follow-up: Where exactly would you draw the line between what a "trusted" tier can and can't bypass, and why there specifically?*

Q87. Design an approach for detecting that your golden dataset itself has become biased or unrepresentative of real production traffic. Periodically sample real production queries (with appropriate privacy handling) and compare their distribution (query type, complexity, topic) against the golden dataset's composition — a growing gap (e.g. production traffic increasingly includes a query pattern the golden dataset has zero examples of) is the signal to actively expand the dataset, rather than assuming static coverage remains adequate indefinitely. *Follow-up: How would you avoid this sampling process itself introducing selection bias into what gets added to the golden dataset?*

Q88. The organization wants "explainability" audits — a non-engineer must be able to review why the system gave a specific answer. Design what that review interface needs, beyond the raw audit log. The raw audit log (SQL executed, sources cited, model versions) is necessary but not sufficient for a non-engineer — you'd need a rendering layer that reconstructs: the original question, the classification decision and its confidence, the actual SQL in a readable form with plain-language explanation of what it computed, each cited source with the specific excerpt that supported the claim, and the guardrail pass/fail status — essentially a structured "explain this answer" view built on top of the audit log's raw data, not a replacement for it. *Follow-up: Should this explainability view be generated at answer-time (and stored) or reconstructed on-demand from the audit log later? What's the tradeoff?*

Q89. Design a rollback plan for a bad deployment of the classifier that's silently misrouting a significant fraction of traffic, discovered 6 hours after deployment.

Immediate: revert the classifier to the previous known-good version (should be a fast, low-

risk operation if versioned properly, per Q22's interface stability argument). Then: audit the 6-hour window's traffic for misrouted requests (identifiable via unusually high SQL rejection rate or low-relevance retrieval correlated with that deployment window) — any user-facing wrong answers from that window may need proactive correction/notification depending on how consequential the underlying decisions were (e.g. if used for actual spend decisions). *Follow-up: What would need to be true about your deployment process for a 6-hour detection gap to be considered acceptable versus a real process failure?*

Q90. Defend the choice to build a custom hybrid orchestrator (explicit steps) versus adopting an off-the-shelf agent framework wholesale, to a skeptical staff engineer who says "you're reinventing LangGraph." The explicit-step design isn't reinventing orchestration primitives from scratch — it's choosing to model this system's *specific* flow (fan-out, merge, correlate, synthesize, guardrail, cache, audit) as first-class, independently-testable steps rather than accepting a framework's generic abstractions for cycles/state that weren't designed around this domain's specific safety and audit requirements (e.g. citation enforcement as a hard gate, fail-closed guardrails). A framework like LangGraph or Step Functions is genuinely the right execution substrate for this (state machine semantics, retries) — the disagreement isn't "framework vs. no framework," it's "generic framework abstractions vs. domain-specific steps built on top of a framework's primitives," and the design doc's own recommendation is the latter. *Follow-up: Where would you draw the line between "use the framework's generic primitives directly" and "build a domain-specific abstraction on top"?*

Tier 5 — Principal (Q91-100)

Q91. You're the technical lead being asked to sign off on this system's readiness for a SOC2 audit. What are the three things you'd check first, and why those three? (1) Audit log immutability — verify the storage layer's actual IAM/permission configuration denies update/delete, not just that the application "doesn't call" those operations, since SOC2 auditors will ask for proof of the control, not a description of intent. (2) Authorization enforcement — verify the fail-closed behavior is actually tested and monitored, not just documented as a design principle. (3) PII handling — verify data minimization at the semantic layer (PII genuinely excluded, not just filtered) has an automated check, not just a one-time manual review, since drift here is a real ongoing risk. These three because SOC2 cares about *provable, ongoing* controls, not documented intentions — the gap between "the design says X" and "we can prove X is actually enforced, continuously" is exactly what an audit probes. *Follow-up: What would you say if the audit log's storage layer technically allows delete via a break-glass admin path — is that acceptable, and under what conditions?*

Q92. A VP asks: "Why does this cost so much more per query than [competitor]'s simpler chatbot?" How do you answer, and what would actually change your answer? The honest framing: cost-per-query isn't comparable in isolation without comparing what's being guaranteed — a simpler chatbot without citation enforcement, layered SQL safety, guardrails, and audit logging is cheaper *because* it's not providing the

correctness/compliance guarantees this system is built for. The answer should be backed by the cost-per-query-by-path instrumentation (Q53/Q79) to show precisely where cost goes, and an honest acknowledgment of which specific guarantees could be relaxed to reduce cost, if the business genuinely doesn't need them for this use case — this is a real tradeoff conversation, not a defense of every design decision as non-negotiable. *Follow-up: Name one specific guarantee in this system you WOULD recommend relaxing if cost pressure were severe enough, and justify it.*

Q93. Design the organizational structure (team ownership boundaries) that would naturally map onto this architecture's control-plane/data-plane split, and explain why that mapping makes sense. Control plane (classifier, policy, cost routing) — owned by a platform/security-adjacent team, since correctness and careful review matter more than raw throughput, and changes here (a new OPA rule, a routing change) have broad blast radius. Data plane (SQL agent, knowledge agent) — owned by domain-specific teams (a data/warehouse team for SQL, an ML/retrieval team for knowledge), since these need deep expertise in their respective execution technologies and scale independently. This mirrors Conway's Law deliberately — the architecture's separation of concerns should map to team boundaries with matching concerns, not fight against natural team structure. *Follow-up: What governance process would you put around changes that need to touch BOTH planes (e.g. a new query path)?*

Q94. The board wants to know: "What's our single biggest architectural risk if we 10x our user base in the next year?" The warehouse's concurrent-query handling under load (Q6/Q52's concerns compounded) is the most likely first bottleneck — the cost estimator protects against any single catastrophic query, but a 10x increase in concurrent moderately-expensive queries stresses a dimension the current per-query cost check doesn't fully address (concurrency, not just individual query cost). Second-order risk: the vector store's namespace/sharding strategy needs to already be in place before 10x growth, not retrofitted after (per the billion-embedding scenario) — retrofitting sharding onto an already-massive single index is disruptive. *Follow-up: If you could only invest engineering effort in solving ONE of these two risks before the 10x growth happens, which would you prioritize and why?*

Q95. A competitor's postmortem reveals their RAG system leaked cross-tenant data through a caching bug. Without knowing their exact bug, walk through how you'd verify this specific class of vulnerability doesn't exist in CDWIA. Audit every cache key construction path (not just the primary result cache — any secondary caching, e.g. embedding caches, rerank score caches per Q58) for whether tenant/scope is included in the key, not just assumed by context. This is a "assume it's broken until proven otherwise" audit, not a "review the design doc and confirm it says scope is included" review — the actual code path needs direct inspection, since a design doc correctly describing scope-safe caching doesn't guarantee every caching layer added later (e.g. a performance optimization added under time pressure) followed the same discipline. *Follow-up: How would you build an automated test that would have caught this class of bug before it shipped, rather than relying on manual audit?*

Q96. You inherit this system from a team that's leaving. What are the first five things you'd personally verify are actually true in production, versus just documented as true in the design doc? (1) Fail-closed policy behavior is actually tested, not just designed. (2) The audit log's immutability is enforced at the storage/IAM layer, not just by convention. (3) The golden dataset is actually current and running in CI, not stale from launch. (4) The circuit breaker thresholds and cost estimator threshold were actually calibrated against real production load, not left at initial defaults. (5) PII data minimization at the semantic layer has been re-verified against the current schema, not just the schema at design time. These five because they're all cases where "the design is correct" and "the design is correctly and currently implemented" can silently diverge over time without anyone noticing, especially across a team transition. *Follow-up: Which of these five would you prioritize verifying in your first week, and what would make you escalate immediately if it failed?*

Q97. Defend the decision to NOT build a fully autonomous, unrestricted agent (free tool use, no allowlist, no validator) when a competitor markets exactly that as "more powerful AI." An unrestricted agent is more capable in principle but has no way to provide the correctness and compliance guarantees (citation enforcement, SQL safety boundaries, immutable audit trail) that make this system trustworthy for actual financial/operational decisions — "more powerful" and "safe to make consequential decisions from" are different axes, and for a FinOps tool specifically, the second axis is the actual product requirement, not a nice-to-have layered on top of raw capability. *Follow-up: Is there a category of query or user where you WOULD relax these restrictions, and what would justify that exception?*

Q98. Design a long-term (2-3 year) evolution plan for this architecture, identifying which components are genuinely durable design decisions versus which are near-term implementation choices likely to be replaced. Durable (unlikely to change regardless of technology shifts): control/data plane separation, layered SQL safety (validation + read-only role + RLS + cost check), citation enforcement as a structural requirement, immutable audit logging — these are architectural principles independent of any specific tool. Near-term implementation choices likely to be replaced: the specific classifier implementation (regex → trained model → possibly a future cheap-LLM-based classifier as costs drop), the specific vector store/reranker vendor, the specific orchestration framework — these are all swappable behind the interfaces the design deliberately keeps stable. *Follow-up: What would have to change in the industry (cost, capability, or regulatory) for one of the "durable" principles to actually need to change?*

Q99. A regulator asks whether this system could be used to make automated decisions with legal/financial consequences without human review. What's your honest answer, and what would need to change to make "yes" a responsible answer? Honest answer: as designed, this is a decision-support tool — it recommends and explains, with citations and confidence scores, but the design doesn't include a human-in-the-loop approval gate for any action based on its output, because it doesn't take actions at all, only answers questions. To responsibly support automated downstream action (e.g. auto-applying a rightsizing recommendation), you'd need to add an explicit approval workflow, tighter confidence thresholds gating what's allowed to auto-execute versus what requires review, and almost certainly a narrower, more heavily-tested scope of actions than the general query-

answering surface this system currently has. *Follow-up: What confidence threshold, if any, would you consider defensible for allowing an automated action without human review, and how would you justify that specific number?*

Q100. Final question — in one paragraph, make the case for why every major design decision in this system traces back to a single underlying principle. What is that principle, and defend it as the thread connecting everything else in this interview. The underlying principle is: **never trust a single layer to be correct, and never let an expensive or risky operation happen before a cheap check has had the chance to stop it.** The control/data plane split (cheap checks before expensive execution), the AST validator plus read-only role plus RLS (three independent layers for the same boundary), citation enforcement plus guardrails (two independent hallucination defenses), the classifier plus LLM fallback (cheap-first, expensive-only-when-needed), the model router's fallback chain (no single provider is a single point of failure), fail-closed policy and cost-estimator behavior (unavailability defaults to safe, not permissive) — every one of these is the same defense-in-depth, fail-safe-by-default pattern applied to a different part of the system. An interviewer probing any individual decision in isolation can always be answered by tracing it back to this principle, which is what makes the architecture coherent rather than a collection of unrelated best practices bolted together.

This is Installment 2. Paste any of your own answers back to me for live evaluation, or ask for Installment 3 (coding questions, system design questions, or the FAANG-style question set).